

HiDiTingV100 OpenHarmony Native Slice 开发

用户指南

文档版本 00B02

发布日期 2025-07-30

前言

概述

本文档介绍了 HiDiTingV100 上 NativeAbilityFwk 子系统的整体框架和进行 Native Slice 应用开发的方法。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B02	2025-07-30	更新 “4.1 SlicePage 跳转方法说明” 。
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 NativeAbilityFwk	1
1.1 NativeAbilityFwk 介绍	1
2 Slice 应用开发	4
2.1 Slice MVP 架构概述	4
2.1.1 Model	4
2.1.2 View	5
2.1.3 Presenter	6
2.2 Slice 开发	6
2.2.1 创建并注册 Slice	6
2.2.2 Slice 生命周期实现建议	9
2.3 SlicePage 开发	9
2.3.1 创建并注册 SlicePage	9
2.3.2 SlicePage 生命周期实现建议	10
3 Slice 跳转	12
3.1 Slice 跳转方法说明	12
3.2 支持 Slice 跳转动效	13
3.2.1 系统自带的 Slice 跳转动效	13
3.2.2 定制 Slice 跳转动效	14
3.2.3 不同配置的动效对 Slice 生命周期的影响	16
3.3 支持 Slice 跟手返回	16
4 SlicePage 跳转	18
4.1 SlicePage 跳转方法说明	18
4.2 支持 SlicePage 跳转动效	19

4.3 支持 SlicePage 跟手返回 19

5 FAQ 21

5.1 如何优化页面跳转时间 21

5.2 涉及屏幕管理的相关接口说明 22

5.3 如何设置当前页面的按键事件响应机制 23

5.4 如何自定义 Slice 的转场动效 23

5.5 如何定制页面的滑动响应 24

5.6 Native 应用开发 Checklist..... 24

Draft

插图目录

图 1-1 Native Ability 框架图 2

图 2-1 MVP RootView 控件树图 5

图 2-2 Slice 注册示例图 8

表格目录

表 2-1 View.h 生命周期接口.....	5
表 2-2 Presenter.h 生命周期接口.....	6
表 2-3 REGIST_SLICE 参数说明	7
表 2-4 REGIST_MENU 参数说明	8
表 2-5 REGIST_MENU_EXT 参数说明	8
表 2-6 REGIST_SLICE_PAGE 参数说明.....	10
表 3-1 ChangeSlice 参数说明.....	13
表 3-2 SwitchSlice 参数说明.....	13
表 3-3 默认动效跳转类型.....	13
表 3-4 TransitionCallback 接口	14
表 3-5 REGIST_TRANSITION 注册参数解析.....	15
表 4-1 NativeAbility::SwitchSliceInPage 参数说明	18
表 5-1 ui_screennotify.h 接口说明	22
表 5-2 power_display_service.h 接口说明.....	22

1

NativeAbilityFwk

1.1 NativeAbilityFwk 介绍

1.1 NativeAbilityFwk 介绍

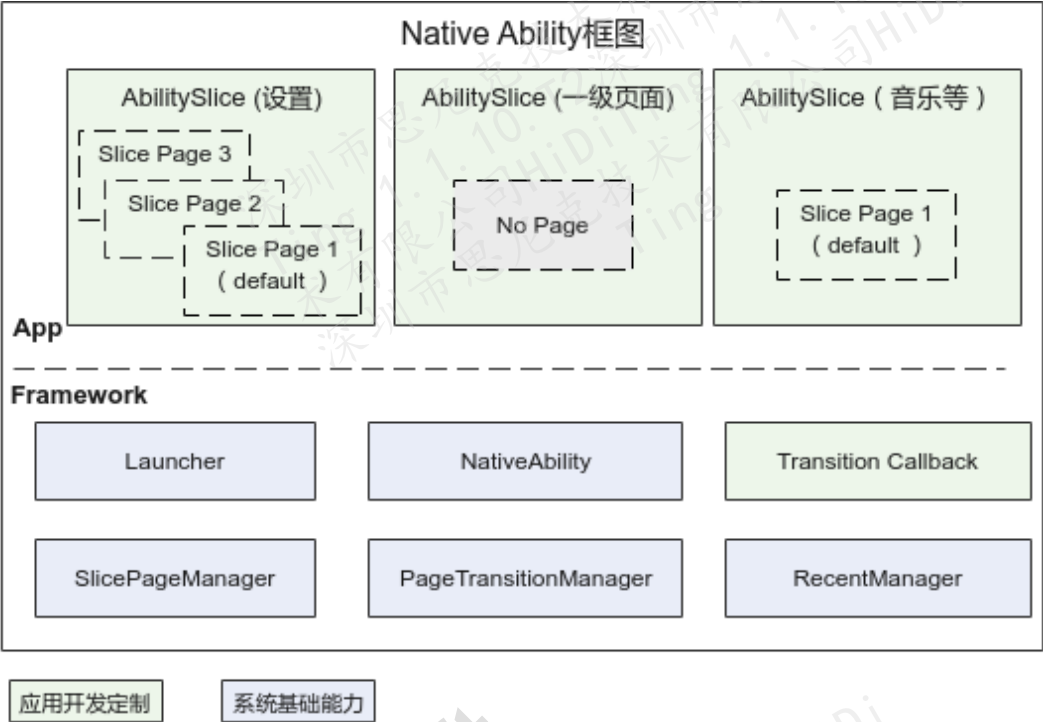
NativeAbilityFwk 子系统是开发 Native Slice 应用的框架，包括 Slice MVP 架构、生命周期管理、转场动效、最近访问记录管理，是基于穿戴产品特点从鸿蒙原生压缩安装的 Native 应用机制定制化而来。

Ability 分 Native Ability 以及 JS Ability 两种：

- Native Ability，用来管理 Native Slice 应用，有且仅有一个实例，使用 C++ 编程；通常一个 Slice 表示一个 Native 应用，如果 Native 应用内部有多个页面，则可以使用多个 SlicePage 实例来一一表示。
- JS Ability，为 JS 应用的入口，一个应用对应一个实例，用于管理 JS Page，JS Page 使用 JS+HML+CSS 编程，具体请参考《HiDiTingV100 OpenHarmony JS 应用开发 用户指南》。

Native Ability 的框架如图 1-1 所示，可以分为 Framework 和 App 两层。

图1-1 Native Ability 框架图



- Framework 层，是 Native Ability 应用开发的框架基础，模块具体职能如下：
 - Launcher：负责 Native 应用资源 and 数据的初始化，以及 Native Slice 的注册。
 - NativeAbility：负责 Native Slice 的创建以及生命周期的管理；并对外提供 Slice 和 SlicePage 跳转的接口。
 - TransitionCallback：转场动效的对外接口，当前在“application\wearable\nativeabilityfwk\src\transition”目录中已提供示例，开发者可以新增定制动效。
 - PageTransitionManager：负责 JS 应用、Native Slice 应用间转场动效的管理。
 - SlicePageManager：负责 Slice 应用内部 SlicePage 的创建以及生命周期的管理。
 - RecentManager：负责记录和截图保存用户最近访问过的 JS 应用、Native Slice 应用。
- App 层，即应用层，当前在“application\wearable\nativeapp”目录中已提供示例，开发者可以实现自己的 Ability Slice 以及 Slice 内部的 Slice Page。
 - Slice 开发可以借助 MVP（可参考“[2.1 Slice MVP 架构概述](#)”）架构模式进行开发。

- 如果 Slice 中的页面较多，可以使用 SlicePage，这样页面的管理和切换可以交由 Framework 层来处理；如果 Slice 中仅有一个页面，可以不使用 SlicePage，严格按照 MVP 架构模式开发。

Draft

2 Slice 应用开发

2.1 Slice MVP 架构概述

2.2 Slice 开发

2.3 SlicePage 开发

2.1 Slice MVP 架构概述

一个完整的 Slice 页面使用 MVP 架构实现，有 Model、View 和 Presenter 三部分，其中 Model 与 View 原则上是解耦的，它们的职责如下：

- M：Model，管理业务模型的数据和业务逻辑。
- V：View，管理显示界面以及与用户的交互。如果 Slice 中包含 SlicePage，SlicePage 需要承担 View 的角色，可参考 SettingView 与 SettingMainPage。
- P：Presenter，负责完成 View 与 Model 之间的交互，数据变化触发更新界面；用户交互触发数据更新。如果一个应用包含较多的 SlicePage，可以考虑将业务逻辑移到 SlicePage 中处理，即 SlicePage 可以承担 Presenter 的角色，可参考 SettingPresenter 与 SettingMainPage。

总之，MVP 是一个比较好的解耦设计，但是在代码简洁与解耦之间，开发者需要做好权衡。

2.1.1 Model

Model 用于管理数据，通常以单例方式存在，以便在多 Slice 间更简单地共享数据。

2.1.2 View

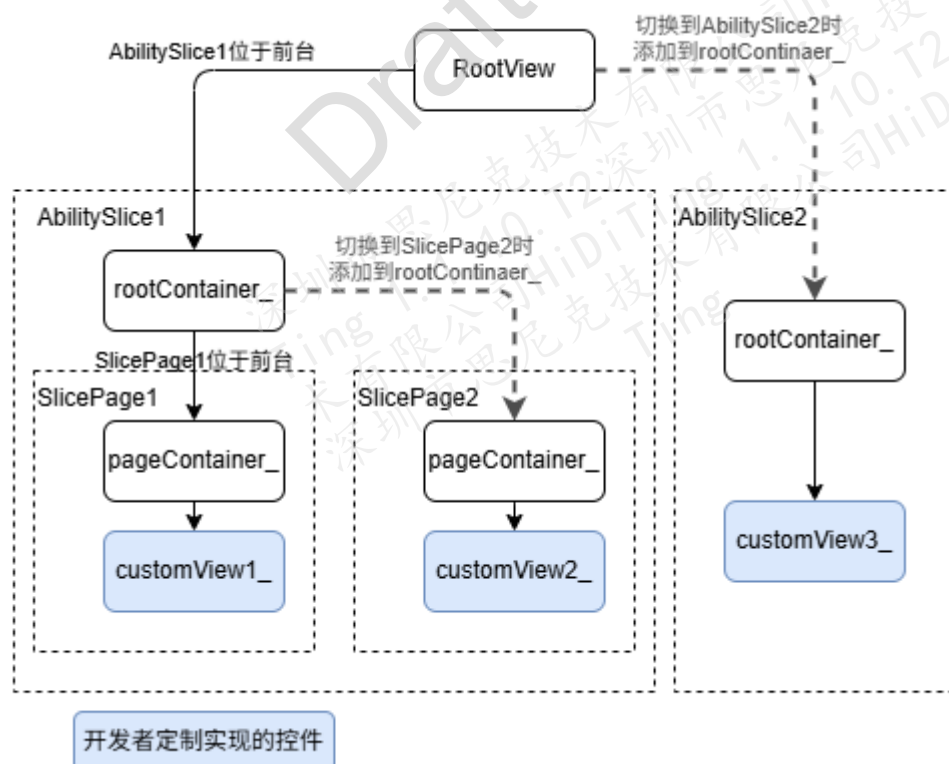
View 继承自 “View.h”，用于视图构建和显示。“View.h” 生命周期需要用户实现的接口如表 2-1 所示。

表2-1 View.h 生命周期接口

接口名	说明
OnStart	启动 Slice 并且在动画（如果有）开始前调用，逻辑实现请参考“2.2.2 Slice 生命周期实现建议”章节。
OnStop	在当前页面不可见并且销毁的时候调用，逻辑实现建议参考“2.2.2 Slice 生命周期实现建议”章节。

View 中的 RootView 控件树嵌套层级如下:

图2-1 MVP RootView 控件树图



须知

不建议将自定义控件 CustomView[x]直接添加到 RootView 中，因为转场动效基于 RootContainer 本身或者截图进行显示，如果将 CustomView[x]直接添加到 RootView 中，则会导致转场动效的显示出现错乱。

2.1.3 Presenter

Presenter 继承自 “Presenter.h”，用于处理数据与视图之间的交互。“Presenter.h” 生命周期需要用户实现的接口如表 2-2 所示。

表2-2 Presenter.h 生命周期接口

接口名	说明
OnStart	启动 Slice 并且在动画（如果有）开始前调用，在 View::OnStart 之后调用，逻辑实现建议参考 “2.2.2 Slice 生命周期实现建议” 章节。
OnResume	动画（如果有）结束并且页面（恢复）可见时调用，逻辑实现建议参考 “2.2.2 Slice 生命周期实现建议” 章节。
OnPause	页面进入后台时调用，逻辑实现建议参考 “2.2.2 Slice 生命周期实现建议” 章节。
OnStop	页面不可见并且销毁时调用，逻辑实现建议参考 “2.2.2 Slice 生命周期实现建议” 章节。

2.2 Slice 开发

2.2.1 创建并注册 Slice

1. 创建 Slice

创建 Slice 有 IDE GUI 工程自动创建和手动创建两种方式，相比于手动创建，IDE GUI 工程自动创建 Slice 优势如下：

- 支持通过拖拉点拽边界 UI 界面。
- 自动创建会自动生成 View、Presenter、Model 的代码。

- 自动创建会生成静态注册的代码，并且注册的 SliceId 在 [VIEW_MAX_INTER_ARRAY_APP, VIEW_MAX_INTER_APP] 之间。

IDE GUI 工程自动创建 Slice，可以参考《HiDiTingV100 HiSparkStudio 使用指南》的“GUI 工程创建与使用”，本文不再赘述。

手动创建 Slice，需要手动添加 View、Presenter、Model 的代码，并手动添加 Slice 静态注册代码，具体可参考 nativeapp 目录下的示例。如果 Slice 内部有 SlicePage，为了代码简洁性，View 和 Presenter 可以简化，并将相关逻辑放到 SlicePage 中。

2. 静态注册 Slice

静态注册 Slice 支持如下两种方式：

- 普通注册：借助宏 REGIST_SLICE 完成，该方式注册的 Slice 可以借助 Slice id 跳转进入 Slice 页面，但是不会在菜单中显示该 Slice 的图标，注册方式为：REGIST_SLICE(id, V, P)
- 菜单注册：借助宏 REGIST_MENU 或者 REGIST_MENU_EXT 完成，该方式注册的应用不仅可以借助 Slice id 进行跳转进入 Slice 页面，也可以在菜单中显示该 Slice 的图标，注册方式为：

REGIST_MENU(id, V, P, icon, iconHexagon, label)：不支持多语言以及图标打包。

REGIST_MENU_EXT(item, V, P)：支持多语言以及图标打包，方便开发者借助 ApplItem 的构造方法进一步扩展。

Slice 应用的注册顺序通过 ID 升序排序，如果用户期望改变菜单中 Slice 应用的排布顺序，可以在

“application\wearable\nativeapp\nativeui\applist\src\ApplistModel.cpp” 的 ResolvingJSApplItems 中定制排布顺序。

表2-3 REGIST_SLICE 参数说明

参数	说明
id	Slice 的 ID，（必须大于 0，0 为无效值）。
V	对应的 View 文件的类名。
P	对应的 Presenter 文件的类名。

表2-4 REGIST_MENU 参数说明

参数	说明
id	Slice 的 ID（必须大于 0，0 为无效值）。
V	对应的 View 文件的类名。
P	对应的 Presenter 文件的类名。
icon	标准应用图标资源文件的路径。
iconHexagon	六边形应用图标资源文件的路径。
label	应用名。

表2-5 REGIST_MENU_EXT 参数说明

参数	说明
item	Appltem 结构体对象。
V	对应的 View 文件的类名。
P	对应的 Presenter 文件的类名。

建议在“XXXPresent.cpp”中进行注册，具体可以参考图 2-2：

图2-2 Slice 注册示例图

```
#include "wearable_log.h"
#include "UiConfig.h"
#include "BluetoothPresenter.h"
#include "BluetoothReconnectView.h"
#include "MainBluetoothView.h"
#include "NativeRegisterManager.h"
#include "UiConfig.h"

namespace OHOS {

    REGIST_SLICE(VIEW_BLUETOOTH_EARPHONE, MainBluetoothView, BluetoothPresenter);

}
```



```
#include "SettingView.h"
#include "SettingModel.h"
#include "SettingPresenter.h"
#include "RebootModel.h"
#include "SetDesktopModel.h"
#include "NativeRegisterManager.h"

namespace OHOS {

REGISTER_MENU(VIEW_SETTING, SettingView, SettingPresenter, PNG_APPLIST_SETTING_IMAGE, PNG_APPLIST_DEFAULT_IMG, "设置");
```

2.2.2 Slice 生命周期实现建议

以 Slice A 转场到 Slice B 为例（不考虑转场动画），具体也可参考“Slice.h/View.h/Presenter.h”的注释。

步骤 1 SliceA OnPause：Slice A 进入 Pause 状态。

Presenter::OnPause：建议暂停动画，清除焦点，清除输入事件监听。

步骤 2 SliceA OnStop：Slice A 进入 Stop 状态。

View::OnStop/~View：建议销毁 Slice A 中自创建的控件资源。（系统会自动将自创建控件从 rootContainer_中移除）

Presenter::OnStop/~Presenter：建议销毁 Slice A 中的非控件资源。

步骤 3 SliceB OnStart：Slice B 进入 Start 状态。

View()/View::OnStart：建议初始化控件资源，并借助接口 GetRootContainer()将其添加到 RootView 控件树中。

Presenter()/Presenter::OnStart：建议初始化非控件资源，基于 Model 非实时数据刷新已初始化的控件状态。

步骤 4 SliceB OnResume：Slice B 进入 Resume 状态。

Presenter::OnResume：建议基于实时数据刷新控件状态、启动页面开场动画、获取焦点、设置输入事件监听等。

----结束

2.3 SlicePage 开发

2.3.1 创建并注册 SlicePage

首先，根据当前 Slice 内部页面的个数和实际业务诉求，决定是否使用 SlicePage。即使只有一个页面，也可以使用 SlicePage。

其次，如果创建 SlicePage，需要新建一个继承于 SlicePageBase 的子类，并在该类的 cpp 文件中将 SlicePage 注册到 Slice 中。如前文所述，SlicePage 可以同时承担 View 和 Presenter 的角色，具体可以参考实例 SettingMainPage 的实现。

注册 SlicePage 使用 REGIST_SLICE_PAGE 方法静态注册到指定的 Slice 中：
REGIST_SLICE_PAGE(sliceId, pageId, V, isMainPage)

表2-6 REGIST_SLICE_PAGE 参数说明

参数	说明
sliceId	Slice 的 ID（必须大于 0，0 为无效值）。
pageId	SlicePage 的 Id（必须大于 0，0 为无效值）。
V	对应的 View 文件的类名。
isMainPage	当前 SlicePage 是否为当前 Slice 的默认 Page，即启动 Slice 是否默认启动到该页面。

2.3.2 SlicePage 生命周期实现建议

以 Slice 内部由 SlicePage A 跳转到 SlicePage B 为例（不考虑动画）：

步骤 1 SlicePage A OnPause：SlicePage A 进入 Pause 状态。如果支持缓存 SlicePage A，则跳过步骤 2。

建议暂停 SlicePage A 中的动画，并清除输入事件监听。

步骤 2 SlicePage A OnStop：SlicePage A 进入 Stop 状态。

建议销毁 SlicePage A 中自创建的控件资源。（系统会自动将自创建控件从 pageContainer_ 中移除）

为了代码简洁，可以承担 Presenter 角色销毁 SlicePage A 的非控件资源。

步骤 3 SlicePage B OnStart：SlicePage B 进入 Start 状态。

建议初始化 SlicePage B 的控件资源，并借助接口 GetSlicePageContainer() 将其添加到 RootView 控件树中。

为了代码简洁，可以承担 Presenter 角色初始化非控件资源，并基于 Model 非实时数据刷新已初始化的控件状态。

步骤 4 SlicePage B OnResume：SlicePage B 进入 Resume 状态。

为了代码简洁，可以承担 Presenter 角色基于实时数据刷新控件状态、启动页面开场动画、获取焦点、设置输入事件监听等。

----结束

当由 Slice A 跳转到 Slice B 的指定 SlicePage B 时，SlicePage B 的生命周期调用与 SliceB Presenter 生命周期调用保持一致。针对这个不同的场景，SlicePage 的实现建议并无区别。

3 Slice 跳转

3.1 Slice 跳转方法说明

3.2 支持 Slice 跳转动效

3.3 支持 Slice 跟手返回

3.1 Slice 跳转方法说明

NativeAbility 单例提供了两种 Slice 跳转的方法：

```
void ChangeSlice(uint16_t targetSliceId, TransitionType type =  
TransitionType::TRANSITION_INVALID,  
uint16_t priority = gSliceDefaultPriority, bool enableSlideBack = false);  
void SwitchSlice(uint16_t sliceId, uint16_t pageId = gPageInvalid,  
TransitionType type = TransitionType::TRANSITION_INVALID, bool enableSlideBack =  
false);
```

// 示例，直接跳转到应用列表

```
NativeAbility::GetInstance().ChangeSlice(VIEW_APPLIST); // 方式1
```

```
NativeAbility::GetInstance().SwitchSlice(VIEW_APPLIST); // 方式2
```

- SwitchSlice 方法：用于跳转到指定 Slice 的匹配 pageId 的 SlicePage 页面，如果 pageId 不指定，则跳转到 Slice 默认页面，等价于 ChangeSlice 方法。Slice 如果无 SlicePage，那么默认页面通过 MVP 中的 View 创建，否则为已经注册的默认 SlicePage 页面。
- ChangeSlice 方法：用于跳转到 Slice 的默认页面，其实现依赖于 SwitchSlice 方法。

无论是 SwitchSlice 还是 ChangeSlice，在快速点击或者 AT 压力测试环境下，如果当前有跳转任务还没有完成，将会将这次新的跳转动作缓存起来。等到当前的跳转任务执行完毕后再延迟执行缓存的跳转动作。

表3-1 ChangeSlice 参数说明

参数	说明
targetSliceId	目标 Slice 的 ID。
type	切换动效类型，默认为 TransitionType::TRANSITION_INVALID，即无动效。
priority	Slice 优先级，默认值为 4，当前未使用。
enableSlideBack	是否支持跟手返回，默认值为 false。

表3-2 SwitchSlice 参数说明

参数	说明
sliceId	目标 Slice 的 ID。
pageId	目标 SlicePage 的 ID。
type	切换动效类型，默认为 TransitionType::TRANSITION_INVALID，即无动效。
enableSlideBack	是否支持跟手返回，默认值为 false。

3.2 支持 Slice 跳转动效

3.2.1 系统自带的 Slice 跳转动效

在头文件 “TransitionType.h” 中定义了系统自定的跳转动效类型，说明如表 3-3 所示。

表3-3 默认动效跳转类型

type	说明
------	----

type	说明
TRANSITION_INVALID	无动效。
TRANSITION_ZOOM	缩放动效（先缩小，后放大）。
TRANSITION_HEXAGONS	蜂窝菜单离场动效。
TRANSITION_BACK_TO_HEXAGONS	蜂窝菜单返场动效。
TRANSITION_ENTER_WATERFALL	瀑布流菜单入场动效。
TRANSITION_BACK_TO_WATERFALL	瀑布流菜单返场动效。
TRANSITION_WATERFALL	瀑布流菜单离场动效。

3.2.2 定制 Slice 跳转动效

- 步骤 1 在 “TransitionType.h” 下添加定制动效 ID。
- 步骤 2 在 transition 目录下，新建定制动效类，该类继承 TransitionCallback 类并重写父类方法。

表3-4 TransitionCallback 接口

类型	接口名	说明
变量	duration_	动效持续时间，不能在类中指定，需要在步骤 3 中指定。
	enableCurrentSnapshot_	当前页面是否截图，不能在类中指定，需要在步骤 3 中指定。
	enableTargetSnapshot_	跳转页面是否截图，不能在类中指定，需要在步骤 3 中指定。
函数	1. virtual void OnTransitionStart(UILImageView * current, UILImageView* target) 2. virtual void OnTransitionStart(UILImageView * current, UI ViewGroup* target) 3. virtual void OnTransitionStart(UI ViewGroup * current, UILImageView* target) 4. virtual void	动效开始调用，用于动效所需控件和状态初始化，四个函数分别对应如下场景： 1. 当前页面截图，跳转页面截图。 2. 当前页面截图，跳转页面不截图。 3. 当前页面不截图，跳转页面截图。

类型	接口名	说明
	OnTransitionStart(UIViewGroup * current, UIViewGroup* target)	4. 当前页面不截图，跳转页面不截图。
	virtual void TransitionAlg(uint32_t time)	动效过程中调用，基于时间对初始化的控件进行状态调整。
	virtual void OnTransitionEnd()	动效结束时调用，根据需要释放动效过程中申请的资源或者状态重置（按需实现）。

步骤 3 在动效源文件中调用 REGIST_TRANSITION 将其注册到 TransitionManager 中。

REGIST_TRANSITION 函数宏原型如下：

```
#define REGIST_TRANSITION(type, func, time, enableCurSnapShot, enableTargetSnapShot) \
    const static OHOS::TransitionRegister (STATIC_VAR)(type, new func(time, \
    enableCurSnapShot, enableTargetSnapShot))
```

例如：

```
REGIST_TRANSITION(TransitionType::TRANSITION_ZOOM, ZoomTransition, 800, true, true);
```

表3-5 REGIST_TRANSITION 注册参数解析

参数	说明
type	动效 ID（在头文件“TransitionType.h”中管理）。
func	动效类名作为入参。
time	动画特效所需的时间。
enableCurSnapShot	是否使能当前 Slice 的截图。
enableTargetSnapShot	是否使能目标 Slice 的截图。

----结束

3.2.3 不同配置的动效对 Slice 生命周期的影响

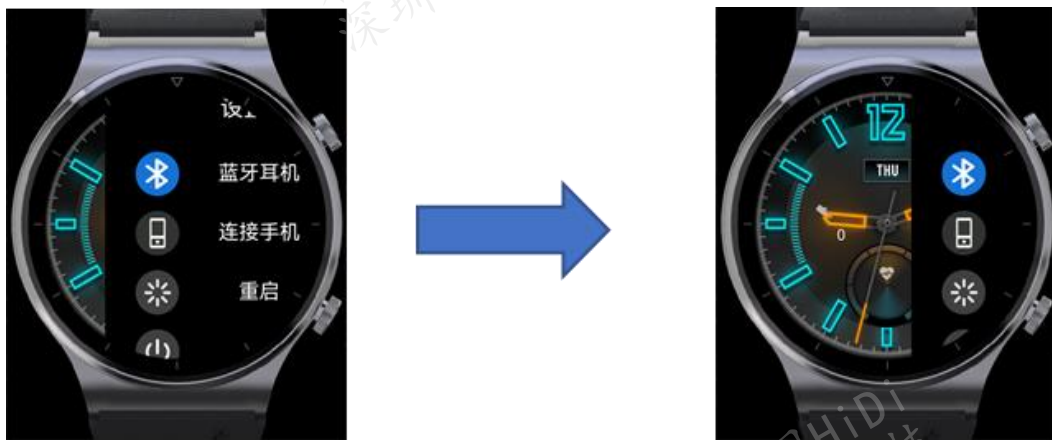
首先，无论跳转是否有动效，Slice 里面各个生命周期的期望行为是保持不变；其次，基于是否使用截图，当前动画一共分为四类，不同的动画，生命周期调用会有些许差异：

- Slice A 使用截图、Slice B 使用截图
Slice A OnPause -> Slice A OnStop -> Slice B OnStart -> Animator -> Slice B OnResume
- Slice A 使用截图、Slice B 不使用截图
Slice A OnPause -> Slice A OnStop -> Slice B OnStart -> Animator -> Slice B OnResume
- Slice A 不使用截图、Slice B 使用截图
Slice A OnPause -> Slice B OnStart -> Animator -> Slice A OnStop -> Slice B OnResume
- Slice A 不使用截图、Slice B 不使用截图
Slice A OnPause -> Slice B OnStart -> Animator -> Slice A OnStop -> Slice B OnResume

更多细节可参考 “application/wearable/nativeabilityfwk/include/TransitionCallback.h” 中的注释。

3.3 支持 Slice 跟手返回

Slice 跟手返回，是指当从 Slice A 切换到 Slice B 后，Slice B 界面可以跟手左右滑动。如果滑动松手时的距离超过屏幕的一半，那么 Slice B 会逐渐退出屏幕，并最终只显示 Slice A 的界面。具体效果可参考下图：



Slice 跟手返回需要将 Slice 跳转方法中的参数 enableSlideBack 设置为 true。

为了支持 Slice 跟手返回，系统会自动在跳转过程中对即将销毁的 Slice 进行截图。另外，也提供对外接口 `SaveSlideBackSnapShotIfNecessary` 供开发者定制截图内容。比如在一级页面下拉菜单中跳转到设置页面后，需要跟手返回到一级页面的表盘界面，此时就需要定制，具体可参考用例 `MainPresenterSample`：

```
void MainPresenterSample::OnPause()
{
    Presenter<MainViewSample>::OnPause();
    // Save watchface content for slide back
    UIView* watchFace = view_ ->GetViewById(WATCH_FACE)->GetParent();
    uint8_t opaScale = watchFace->GetOpaScale();
    watchFace->SetOpaScale(OPA_OPAQUE);
    PageTransitionMgr::GetInstance().SaveSlideBackSnapShotIfNecessary(watchFace);
    watchFace->SetOpaScale(opaScale);
    mainPresenterState_ = MainPresenterState::PAUSE;
}
```

说明

- 以下情况会导致历史截图被删除，进而不能够跟手返回：
 - 没有连续设置跟手返回，例如从 A 进入 B 设置跟手返回、B 进入 C 不设置跟手返回，那么 A 的截图将会被删除。
 - 跟手返回最多只支持 3 张截图，例如从 A 进入 B，B 进入 C，C 进入 D 都设置了跟手返回，当前在 D 能够跟手返回到 C。但如果从 D 进入到 E，将会删除历史截图，在 E 界面不能跟手返回，只能通过其他方式跳转。
- 只能跟手返回到 native 应用。
- 由于当前 Slice 的根容器控件通过监听拖拽事件进行跟手返回，当子控件同时监听拖拽事件的时候就会存在滑动冲突的问题，具体场景和对应方案如下：
 - 子控件内容支持水平方向或竖直方向滑动（如 `UICollectionViewController`、`UISwipeView` 及 `UIScrollView`），建议客户根据需要使用 `UICollectionViewControllerNested`、`UISwipeViewNested` 和 `UIScrollViewNested` 替代相应的类，具体参考 Demo 代码 `SettingView.cpp`、`SetDesktopView.cpp`。
 - 客户 Slice 内部嵌套多级页面，支持右滑返回上一级。如果多级子页面监听根容器的右滑事件，会与 Slice 跟手返回冲突。建议客户将 Slice 中多级页面的右滑监听配到内部子页面，并拦截事件分发。

4 SlicePage 跳转

- 4.1 SlicePage 跳转方法说明
- 4.2 支持 SlicePage 跳转动效
- 4.3 支持 SlicePage 跟手返回

4.1 SlicePage 跳转方法说明

SlicePage 跳转是指 Slice 内部 SlicePage 间的跳转，可以使用如下两种跳转方式：

- 使用 NativeAbility 提供的 SwitchSliceInPage 方法跳转到另一个 SlicePage。

```
bool SwitchSliceInPage(uint16_t pageId, void* data = nullptr,  
    TransitionType type = TransitionType::TRANSITION_INVALID, bool canBack = false);
```

表4-1 NativeAbility::SwitchSliceInPage 参数说明

参数	说明
pageId	目标 SlicePage 的 ID。
data	用于 SlicePage 间的数据传递，传递过程中不会做 data 备份
type	跳转动效类型，默认为 TransitionType::TRANSITION_INVALID，即无动效。动效可复用不使用截图的 TransitionCallback 实例。
canBack	是否缓存当前页面，并支持跟手返回，默认值为 false。

如果 Slice 正在切换或者 Slice 内部的 SlicePage 正在切换，则该操作会被取消，并返回 false。

如果不缓存页面，请不要在该函数调用后继续访问当前 slice 的成员变量，因为当前 slice 会在该函数中销毁。

- 使用 NativeAbility 提供的 BackToPrevSlicePage 方法返回到上一个缓存的 SlicePage：

```
bool BackToPrevSlicePage();
```

如果 Slice 正在切换、Slice 内部的 SlicePage 正在切换或者没有缓存的 Slice Page，则该操作会被取消，并返回 false。

请不要在该函数调用后继续访问当前 slice 的成员变量，因为当前 slice 会在该函数中销毁。

4.2 支持 SlicePage 跳转动效

SlicePage 的跳转动效可复用不使用截图的 Slice 跳转动效，即参数 enableCurrentSnapShot 和 enableTargetSnapShot 均为 false 的 TransitionCallback 实现类。开发者可以参考“[3.2 支持 Slice 跳转动效](#)”章节定制 SlicePage 跳转动效。

在有跳转动效的场景，为了保证跳转过程中当前页面还能够实时变化，Slice 内部 SlicePage 跳转的生命周期会有差异。以 SlicePage A 跳转到 SlicePage B 为例，生命周期如下：

SlicePage B OnStart -> Animator -> SlicePage A OnPause -> SlicePage A OnStop -> SlicePage B OnResume

4.3 支持 SlicePage 跟手返回

SlicePage 可以在跳转的时候指定是否缓存，如果缓存（参数 canBack 为 true），那么在跳转离开后仍能够跟手返回到当前 SlicePage。

SlicePage 跟手返回有如下约束点需要注意：

- 未限制可缓存的 SlicePage 的数量，因此开发者在使用该功能的时候需考虑整个系统内存的限制。
- 如果 Slice 内部 SlicePage 的跳转不需要缓存，则会清空当前 Slice 内部已缓存的 SlicePage。
- 如果跳转离开当前 Slice，当前 Slice 内部的 SlicePage 缓存也会被清空。

- 如果 SlicePage 既要跟手返回到前一个页面，又要支持内部滑动，需要参考“[3.3 支持 Slice 跟手返回](#)”章节妥善处理嵌套滑动问题。

Draft

5 FAQ

- 5.1 如何优化页面跳转时间
- 5.2 涉及屏幕管理的相关接口说明
- 5.3 如何设置当前页面的按键事件响应机制
- 5.4 如何自定义 Slice 的转场动效
- 5.5 如何定制页面的滑动响应
- 5.6 Native 应用开发 Checklist

5.1 如何优化页面跳转时间

在页面跳转过程中，图片资源加载往往耗时最多，因此可以通过减少前期图片加载的时间来优化减少页面跳转时间，具体方法如下：

1. 在 OnStart 阶段建议一次性加载第一帧页面显示所需的图片；如果第一帧页面所需的图片资源较多，可以将这些图片资源单独打包为一个 bin 文件（请参考《HiDiTingV100 UIkit 开发指南》的“图片资源打包”章节）。对于序列帧场景，由于在 OnStart 阶段仅需加载第一帧的图片资源，则不需要单独打包。
2. 在 OnResume 阶段借助 PostGraphicEvent 方法延迟加载剩余的图片资源。注意，延迟访问是为了防止内存释放后的野指针问题，代码实现可参考下述示例：

```
{  
    std::weakptr<bool> wk = isExits_; // isExits_为当前类的私有成员变量，类型为  
std::shared_ptr<bool>  
    GraphicService::GetInstance()->PostGraphicEvent([this, wk]() {  
        if (wk.expired()) {  
            return; //当前类已经销毁
```

```

    }
  });
}

```

5.2 涉及屏幕管理的相关接口说明

设置灭屏后的 Slice 的存活期，灭屏时长超过存活期时，再次亮屏会跳回默认界面，默认界面是 **VIEW_MAIN_SAMPLE**。

表5-1 ui_screennotify.h 接口说明

接口名	说明
set_back_to_home_in_ternval	设置存活期时间，单位：s，默认是 5 秒。 设置灭屏后 Slice 的存活期，需要在 Slice 对应的 Presenter 的 OnStart 方法中调用，设置该 Slice 的灭屏后的存活时长。
get_back_to_home_in_ternval	获取存活期时间，单位：s。

设置某个 Slice 亮屏时间，超过设置时间后，屏幕灭屏。

表5-2 power_display_service.h 接口说明

接口名	说明
set_screen_set_keepon_timeout	设置亮屏时间，单位：ms。 使用示例，如果需要设置某个 Slice 亮屏时间，可以在 Slice 对应的 Presenter 的 OnStart 方法里面添加如下代码。 <pre> const power_display_svr_api_t *display_api = power_display_svr_get_api(); if (display_api->get_screen_state() != SCREEN_ON) { display_api->turn_on_screen(); } //当设置的值是-1 的时候，设置屏幕常亮 display_api->set_screen_set_keepon_timeout(-1); //设置屏幕亮屏 5 秒后灭屏 display_api->set_screen_set_keepon_timeout(5000); //设置立刻灭屏 </pre>

接口名	说明
	<code>set_screen_set_keepon_timeout(0);</code>

5.3 如何设置当前页面的按键事件响应机制

当前的 MVP 架构实现中已经配置了默认按键事件响应类 `KeyInputListener`：

- 在 `AbilitySlice::OnResume` 中配置默认事件响应类。
- 在 `AbilitySlice::OnPause` 方法中清除事件响应类，保证在页面跳转的过程中不处理按键事件响应。

因此，有两种方法：

- 直接修改默认事件响应类 `KeyInputListener`，根据 `Slice` 的不同来处理按键事件。
- 在 `Presenter::OnResume/OnPause` 方法或者 `SlicePage::OnResume/OnPause` 方法中，重新配置当前页面的自定义事件响应类。

5.4 如何自定义 Slice 的转场动效

自定义 `Slice` 的转场动效简单来说，就是重写 `TransitionCallback` 类，不同的动画类型使用不同的静态注册方法。

以蜂窝转场动效 `HexagonsTransition` 为例：

步骤 1 根据转场的要求，明确是蜂窝页面以及目标页面是否需要截图。

由于蜂窝界面需要对内部的图标做变换，所以不需要截图；而目标页面是个整体，可以使用截图。

虽然转场使用截图会占用资源，但是性能会好，最终的选择需要结合效果、内存以及性能综合来看。

步骤 2 在 “`TransitionType.h`” 中添加新的转场类型：`TRANSITION_HEXAGONS`。

步骤 3 重写 `TransitionCallback` 类：`class HexagonsTransition : public TransitionCallback`

`OnTransitionStart`：动画开始阶段，对控件（`Slice` 的截图控件或者 `root container`）进行预处理。

TransitionAlg: 动画进行阶段, 对控件做 matrix 变换或者 alpha 处理, 已达到转场的效果。

OnTransitionEnd: 动画结束阶段, 重置状态。

步骤 4 静态注册转场类型, 保证后续可以直接使用。

```
REGIST_TRANSITION(TransitionType::TRANSITION_HEXAGONS,  
HexagonsTransition, 300, false, true);
```

- 参数 300: 转场持续时间。
- 参数 false: 源界面不需要截图。
- 参数 true: 目标界面需要截图。

----结束

5.5 如何定制页面的滑动响应

如果当前页面配置支持跟手返回, 系统会为当前页面配置跟手返回相应的滑动响应监听。

开发者也可以自定义滑动响应, 如果系统默认已配置, 则需要重写 Presenter 或者 SlicePage 的 OnResume 方法来覆盖系统默认配置。具体实现可以分为如下两个场景:

- 当前 Slice 无 SlicePage: 在 Presenter 的实现子类 OnResume 方法中自定义 rootContainer_ 的跟手滑动监听
view_ -> GetRootContainer() -> SetOnDragListener(this)
- 当前 Slice 有 SlicePage: 在 SlicePage 的实现子类 OnResume 方法中自定义 pageContainer_ 的跟手滑动监听
pageContainer_ -> SetOnDragListener(dragListener)

5.6 Native 应用开发 Checklist

由于部分应用存在生命周期不合理, 根容器使用不规范和冗余代码较多等问题, 因此总结了以下 Checklist, 用户可在开发应用时对照检查:

步骤 1 检查并确认是否存在非 UI 线程修改 UI 控件的问题, 常见的场景有:

- 在 OSTimer 的 Callback 中直接操作 UI, 建议使用 Animator 或者 Task 替换实现。

- 在 MsgCenterNotifyProc 中操作直接 UI，建议配合 PostGraphicEvent 来实现，但需谨慎处理因延迟执行而导致的空指针或者野指针问题，可以借助 shared_ptr 和 weak_ptr 来解决，类似“5.1 如何优化页面跳转时间”章节的处理。

步骤 2 检查并删除冗余代码：

- 删除自实现应用中关于 KeyEventListener 的配置，因为已在 AbilitySlice::OnResume 方法中配置，如需定制，可参考“4.1 SlicePage 跳转方法说明”章节内容。
- 删除自实现应用中关于 RootView::RemoveAll 的调用，因为已在 View::TearDownView 方法中统一处理。注意，请谨慎删除非 OnStop 和销毁阶段的 RemoveAll 函数调用。
- 删除 XXXPresenter 冗余的继承关系，即未重写函数或者重写为空方法体的基类。
- 删除 XXXView 和 XXXPresente 关于生命周期的日志代码，因为 AbilitySlice 文件已经添加了通用 Log。
 - 删除 OnStart/OnStop 日志
 - 删除 OnPause/OnResume 日志
- 删除 XXXView 和 XXXPresenter 关于基类方法的冗余调用，因为基类的方法体为空实现。
 - 删除基类 View<P>::OnStart|View<P>::OnStop 冗余调用
 - 删除基类 Presenter<V>::OnStart|Presenter<V>::OnPause|Presenter<V>::OnResume|Presenter<V>::OnStop 冗余调用

步骤 3 检查并确保无 SlicePage 的 Slice 应用使用 GetRootContainer()->Add 方法将页面内的控件添加到 RootView 控件树中，而不是使用 RootView->Add 方法。

步骤 4 检查并确保 SlicePage 使用 GetSlicePageContainer()->Add 方法将页面内的控件添加到 RootView 控件树中，而不是使用 RootView->Add 方法。

步骤 5 检查并确保 OnPause 阶段清空 Focus，OnResume 阶段恢复 Focus，从而保证有动画转场过程中 Focus 时序不乱。

- 删除在页面销毁过程（OnStop 或者析构）中的 ClearFocus 操作，因为 AbilitySlice::OnPause 方法会统一处理。
- 将 RequestFocus 操作放到 Presenter 或者 SlicePage 子类的 OnResume 方法中。

步骤 6 检查并确保 View 或者 SlicePage 子类在 OnStart 阶段已完成当前待显示页面的初始化，从而保证转场动画可以拿到完整内容做动画。

步骤 7 检查并确保 Presenter 或者 SlicePage 子类在 OnResume 阶段基于实时数据刷新当前页面的 UI、启动页面开场动画、获取焦点、设置输入事件监听等，保证转场动画结束后才执行这些动作。

步骤 8 检查并确保 View 或者 SlicePage 子类的 OnStart 实现逻辑不依赖于 Presenter 子类的 OnStart 逻辑，因为 Presenter::OnStart 方法的执行晚于 View 或者 SlicePage 子类的 OnStart 方法执行。

----结束

Draft